


```

import numpy as np

# -----
# 1. q-Pochhammer and a_2(n1,n2;q) from a tiny RPP enumeration
# -----

def q_poch(n, q):
    """(q)_n = product_{k=1}^n (1 - q^k), n >= 0."""
    if n <= 0:
        return 1.0
    prod = 1.0
    for k in range(1, n+1):
        prod *= (1.0 - q**k)
    return prod

def a2(n1, n2, q):
    """
    Approximate a_2(n1,n2;q) up to an overall factor.

    r=2 => staircase  $\delta_3$  with boxes:
    A=(1,1), B=(1,2), C=(2,1)
    outer diagonal: (B,C) = (n1,n2)
    inner: A = x  $\in$  [0, min(n1,n2)].

    Labelle-type weight (up to global factor C(n1,n2;q)):
     $\sum_x q^{x^2} / [(q)_x^2 (q)_{n1-x} (q)_{n2-x}]$ 
    Global factors cancel in L_2 ratios, so we ignore them.
    """
    s = 0.0
    m = min(n1, n2)
    for x in range(m+1):
        denom = (q_poch(x, q)**2) * q_poch(n1 - x, q) * q_poch(n2 - x, q)
        s += (q**(x*x)) / denom
    return s

# -----
# 2. Build truncated generator L_2 on {0,...,N_max}^2
# -----

def build_L2(N_max, q):
    """
    Build generator matrix L_2 on state space S = {(n1,n2)} with 0 <= n1,n2
    Transitions:
    (n1,n2) -> (n1-1,n2) with rate r1 = q^{n2-n1} * a2(n1-1,n2)/a2(n1,n2)
    (n1,n2) -> (n1,n2-1) with rate r2 = q^{-n2} * a2(n1,n2-1)/a2(n1,n2)
    (This is the r=2 specialization of L_r from the paper.)
    """
    # Precompute a2 table
    a_tab = np.zeros((N_max+1, N_max+1), dtype=float)

```

```

for n1 in range(N_max+1):
    for n2 in range(N_max+1):
        a_tab[n1, n2] = a2(n1, n2, q)

# Map (n1,n2) -> flat index
def idx(n1, n2):
    return n1*(N_max+1) + n2

dim = (N_max+1)**2
L = np.zeros((dim, dim), dtype=float)

for n1 in range(N_max+1):
    for n2 in range(N_max+1):
        k = idx(n1, n2)
        a_cur = a_tab[n1, n2]
        if a_cur == 0.0:
            continue # unreachable / degenerate; skip

        # i = 1 jump: n1 -> n1 - 1
        if n1 > 0:
            a_prev = a_tab[n1-1, n2]
            if a_prev > 0.0:
                rate = (q**(n2 - n1)) * (a_prev / a_cur)
                k2 = idx(n1-1, n2)
                L[k, k2] += rate
                L[k, k] -= rate

        # i = 2 jump: n2 -> n2 - 1
        if n2 > 0:
            a_prev = a_tab[n1, n2-1]
            if a_prev > 0.0:
                rate = (q**(-n2)) * (a_prev / a_cur)
                k2 = idx(n1, n2-1)
                L[k, k2] += rate
                L[k, k] -= rate

    return L

# -----
# 3. Spectral gap of L_2
# -----

def spectral_gap(L, tol=1e-10):
    """
    For generator matrix L, eigenvalues  $\lambda$  have  $\text{Re}(\lambda) \leq 0$ .
    Spectral gap = smallest positive  $-\text{Re}(\lambda)$  excluding 0.
    """
    vals = np.linalg.eigvals(L)
    # real parts (generator should be real, but numeric noise)
    re = np.real(vals)
    # sort by Re

```

```

    re_sorted = np.sort(re)
    # largest (closest to 0) is ~0, next is negative; take smallest |neg|
    # safer: find negative real parts whose magnitude > tol
    neg = -re[re < -tol]
    if neg.size == 0:
        return 0.0
    return float(np.min(neg))

# -----
# 4. Run: choose q and N_max, print gap
# -----

if __name__ == "__main__":
    q = 0.7          # deformation parameter, 0<q<1
    N_max = 4        # outer-diagonal cutoff

    L = build_L2(N_max, q)
    gap = spectral_gap(L)

    print(f"q = {q}, N_max = {N_max}")
    print(f"spectral gap (approx) m_q ≈ {gap:.6e}")

```

```

import numpy as np

# -----
# Assume you already have:
#   q_racah_jacobi_matrix
#   doob_transform
#   spectral_gap
# -----

# ---- 1. q-Racah overlap kernel R(x,y) (simple truncated version) ----
def q_racah_kernel(x_vals, y_vals, q):
    """
    Build a discrete q-Racah overlap kernel matrix:
        R_{ij} = R_r(x_i, y_j; q)
    We use a simplified symmetric version: exp(-|x-y| * (1-q)).
    Replace later with true {}_4phi_3 when needed.
    """
    R = np.zeros((len(x_vals), len(y_vals)))
    for i,x in enumerate(x_vals):
        for j,y in enumerate(y_vals):
            R[i,j] = np.exp(-(abs(x-y))*(1-q))
    # normalize rows
    R = R / R.sum(axis=1, keepdims=True)
    return R

# ---- 2. Wilson-loop multiplication W(chi) ----
def wilson_operator(chi_vals, T=1):

```

```

def wilson_operator(chi_vals, I=1):
    """
    Wilson operator W_I acting as multiplication:
        W f(chi) = p_I(chi + chi^{-1}) f(chi)
    Use p_I(z) = z^I as placeholder; can replace with Chebyshev/trace-polyno
    """
    z = chi_vals + 1.0/chi_vals
    W = np.diag(z**I)
    return W

# ---- 3. Boundary→bulk projection Lambda (simple) ----
def projection_kernel(chi_vals, N):
    """
    Simple version of Lambda: embeds boundary states into bulk Doob states.
    Use a normalized Gaussian-like projection for now.
    Replace with true RPP projection when ready.
    """
    Lambda = np.zeros((N+1, len(chi_vals)))
    for i in range(N+1):
        for j,chi in enumerate(chi_vals):
            Lambda[i,j] = np.exp(-(i - 2*chi)**2 / 10.0)
    # Normalize columns
    Lambda = Lambda / Lambda.sum(axis=0, keepdims=True)
    return Lambda

# ---- 4. Build full composite operator T_q ----
def build_Tq(N, q, chi_vals):
    # Step 1: bulk Doob generator exponential
    H = q_racah_jacobi_matrix(N, q, 1.0,1.0,1.0,1.0)
    Q,_,_ = doob_transform(H)
    T_bulk = np.real_if_close(np.linalg.expm(Q)) # exp(Q)

    # Step 2: boundary Wilson-loop operator
    W = wilson_operator(chi_vals, I=1)

    # Step 3: boundary change-of-basis kernel
    R = q_racah_kernel(chi_vals, chi_vals, q)

    # Step 4: projection Lambda
    Lambda = projection_kernel(chi_vals, N)
    Lambda_T = Lambda.T

    # Full operator:
    # T = Lambda^T * T_bulk * Lambda * R * W
    T = Lambda_T @ T_bulk @ Lambda @ R @ W
    return T

# ---- 5. Scan spectrum of T_q ----
def analyze_Tq(N, q):
    chi_vals = np.linspace(1.1, 2.0, 10)
    T = build_Tq(N, q, chi_vals)

```

```

    eigvals = np.linalg.eigvals(T)
    eigvals = np.real_if_close(eigvals)
    # Sort by magnitude descending
    eigvals_sorted = np.sort(np.abs(eigvals))[:, -1]
    gap = eigvals_sorted[0] - eigvals_sorted[1]
    return eigvals_sorted, gap

# RUN DEMO
N = 8
q = 0.92
eigvals, gap = analyze_Tq(N, q)
print("Eigenvalues of T_q magnitude:", eigvals)
print("Transfer-matrix gap:", gap)

```

```

#=====
# GPU-JAX: 4D Viscous Hamilton-Jacobi Flow (Correct Semigroup)
#=====

import jax
import jax.numpy as jnp
from jax import jit

#-----
# Grid setup (4D)
#-----
L = 24          # use L=16 or 24 for your first run; 32 is heavy
xmax = 2.0
dx = 2*xmax / L

x = jnp.linspace(-xmax, xmax, L)
X, Y, Z, W = jnp.meshgrid(x, x, x, x, indexing="ij")
grid = jnp.stack([X, Y, Z, W], axis=-1)  # (L,L,L,L,4)

#-----
# Strongly convex quadratic potential:
#
#  $S_0(x) = 0.5 * x^T H x$ 
#
# IMPORTANT: we use broadcasting, not vmap.
#-----

Hmat = jnp.array([
    [4.0, 0.3, 0.1, 0.0],
    [0.3, 3.0, 0.2, 0.1],
    [0.1, 0.2, 5.0, 0.1],
    [0.0, 0.1, 0.1, 3.5]
])

#  $S_0(x) = 0.5 * \sum_{i,j} H_{ij} x_i x_j$  using Einstein summation

```

```

S = 0.5 * jnp.einsum('...i,ij,...j->...', grid, Hmat, grid)

#-----
# 4D Laplacian and gradient magnitude
#-----

def laplace4(F):
    return (
        -8*F
        + jnp.roll(F, 1, axis=0) + jnp.roll(F, -1, axis=0)
        + jnp.roll(F, 1, axis=1) + jnp.roll(F, -1, axis=1)
        + jnp.roll(F, 1, axis=2) + jnp.roll(F, -1, axis=2)
        + jnp.roll(F, 1, axis=3) + jnp.roll(F, -1, axis=3)
    ) / (dx*dx)

def grad2_4(F):
    Fx = (jnp.roll(F, -1, axis=0) - jnp.roll(F, 1, axis=0)) / (2*dx)
    Fy = (jnp.roll(F, -1, axis=1) - jnp.roll(F, 1, axis=1)) / (2*dx)
    Fz = (jnp.roll(F, -1, axis=2) - jnp.roll(F, 1, axis=2)) / (2*dx)
    Fw = (jnp.roll(F, -1, axis=3) - jnp.roll(F, 1, axis=3)) / (2*dx)
    return Fx*Fx + Fy*Fy + Fz*Fz + Fw*Fw

#-----
# HJ time stepping
#-----

@jit
def hj_step(S, dt):
    return S + dt * (laplace4(S) - grad2_4(S))

#-----
# Curvature extraction at center
#-----

def curvature_center(S):
    c = L//2
    Sxx = (S[c+1,c,c,c] - 2*S[c,c,c,c] + S[c-1,c,c,c])/(dx*dx)
    Syy = (S[c,c+1,c,c] - 2*S[c,c,c,c] + S[c,c-1,c,c])/(dx*dx)
    Szz = (S[c,c,c+1,c] - 2*S[c,c,c,c] + S[c,c,c-1,c])/(dx*dx)
    Sww = (S[c,c,c,c+1] - 2*S[c,c,c,c] + S[c,c,c,c-1])/(dx*dx)
    return float((Sxx + Syy + Szz + Sww)/4.0)

#-----
# Run evolution
#-----

dt = 0.0005
steps = 300

print("step    curvature_trace")

```

```

for t in range(steps):
    S = hj_step(S, dt)

    if t % 30 == 0:
        curv = curvature_center(S)
        print(f"{t:4d}    {curv:.6f}")

```

```

#=====
# GPU-JAX: 4D Viscous Hamilton-Jacobi Flow (Correct Semigroup)
#=====

import jax
import jax.numpy as jnp
from jax import jit

#-----
# Grid setup (4D)
#-----
L = 24          # use L=16 or 24 for your first run; 32 is heavy
xmax = 2.0
dx = 2*xmax / L

x = jnp.linspace(-xmax, xmax, L)
X, Y, Z, W = jnp.meshgrid(x, x, x, x, indexing="ij")
grid = jnp.stack([X, Y, Z, W], axis=-1)    # (L,L,L,L,4)

#-----
# Strongly convex quadratic potential:
#
#  $S_0(x) = 0.5 * x^T H x$ 
#
# IMPORTANT: we use broadcasting, not vmap.
#-----

Hmat = jnp.array([
    [4.0, 0.3, 0.1, 0.0],
    [0.3, 3.0, 0.2, 0.1],
    [0.1, 0.2, 5.0, 0.1],
    [0.0, 0.1, 0.1, 3.5]
])

#  $S_0(x) = 0.5 * \sum_{i,j} H_{ij} x_i x_j$  using Einstein summation
S = 0.5 * jnp.einsum('...i,ij,...j->...', grid, Hmat, grid)

#-----
# 4D Laplacian and gradient magnitude
#-----

```

```

def laplace4(F):
    return (
        -8*F
        + jnp.roll(F, 1, axis=0) + jnp.roll(F, -1, axis=0)
        + jnp.roll(F, 1, axis=1) + jnp.roll(F, -1, axis=1)
        + jnp.roll(F, 1, axis=2) + jnp.roll(F, -1, axis=2)
        + jnp.roll(F, 1, axis=3) + jnp.roll(F, -1, axis=3)
    ) / (dx*dx)

def grad2_4(F):
    Fx = (jnp.roll(F, -1, axis=0) - jnp.roll(F, 1, axis=0)) / (2*dx)
    Fy = (jnp.roll(F, -1, axis=1) - jnp.roll(F, 1, axis=1)) / (2*dx)
    Fz = (jnp.roll(F, -1, axis=2) - jnp.roll(F, 1, axis=2)) / (2*dx)
    Fw = (jnp.roll(F, -1, axis=3) - jnp.roll(F, 1, axis=3)) / (2*dx)
    return Fx*Fx + Fy*Fy + Fz*Fz + Fw*Fw

#-----
# HJ time stepping
#-----

@jit
def hj_step(S, dt):
    return S + dt * (laplace4(S) - grad2_4(S))

#-----
# Curvature extraction at center
#-----

def curvature_center(S):
    c = L//2
    Sxx = (S[c+1,c,c,c] - 2*S[c,c,c,c] + S[c-1,c,c,c])/(dx*dx)
    Syy = (S[c,c+1,c,c] - 2*S[c,c,c,c] + S[c,c-1,c,c])/(dx*dx)
    Szz = (S[c,c,c+1,c] - 2*S[c,c,c,c] + S[c,c,c-1,c])/(dx*dx)
    Sww = (S[c,c,c,c+1] - 2*S[c,c,c,c] + S[c,c,c,c-1])/(dx*dx)
    return float((Sxx + Syy + Szz + Sww)/4.0)

#-----
# Run evolution
#-----

dt = 0.0005
steps = 300

print("step    curvature_trace")

for t in range(steps):
    S = hj_step(S, dt)

    if t % 30 == 0:
        curv = curvature_center(S)
        print(f"{t:4d}    {curv:.6f}")

```

```

step    curvature_trace
0      4.200736
30     3.713531
60     3.330270
90     3.020311
120    2.764121
150    2.548645
180    2.364778
210    2.205960
240    2.067349
270    1.945289

```

```

import numpy as np

# Your recorded data
t_vals = np.array([0, 30, 60, 90, 120, 150, 180, 210, 240, 270], dtype=float)
lam_vals = np.array([4.200736, 3.713531, 3.330270, 3.020311, 2.764121,
                    2.548645, 2.364778, 2.205960, 2.067349, 1.945289], dtype=

# Compute 1/lambda
inv_lam = 1.0 / lam_vals

# Fit  $\alpha$  from slope of 1/lambda vs t
A = np.vstack([t_vals, np.ones_like(t_vals)]).T
alpha, intercept = np.linalg.lstsq(A, inv_lam, rcond=None)[0]

print("Estimated Riccati coefficient  $\alpha$  =", alpha)
print("Intercept ( $1/\lambda_0$ ) =", intercept)
print("Prediction:  $\lambda(t) \approx 1 / (\text{intercept} + \alpha t)$ ")

```

```

Estimated Riccati coefficient  $\alpha$  = 0.0010214540013711412
Intercept ( $1/\lambda_0$ ) = 0.23878515618101043
Prediction:  $\lambda(t) \approx 1 / (\text{intercept} + \alpha t)$ 

```

```

pred = 1.0 / (intercept + alpha * t_vals)

print("t       $\lambda_{\text{true}}$        $\lambda_{\text{pred}}$ ")
for t, lam, p in zip(t_vals, lam_vals, pred):
    print(f"{t:6.1f}    {lam:8.4f}    {p:8.4f}")

```

```

t       $\lambda_{\text{true}}$        $\lambda_{\text{pred}}$ 
0.0    4.2007    4.1879
30.0    3.7135    3.7116
60.0    3.3303    3.3325
90.0    3.0203    3.0237
120.0    2.7641    2.7673
150.0    2.5486    2.5510
--- -    - - - - -    - - - - -

```

180.0	2.3648	2.3660
210.0	2.2060	2.2061
240.0	2.0673	2.0664
270.0	1.9453	1.9433

```

=====
# GPU-JAX: Full 4x4 Hessian Eigenvalues for 4D HJ Curvature Flow
#
#  $dS/dt = \Delta S - |\nabla S|^2$  (viscous Hamilton-Jacobi semigroup)
#
# This version:
# * Evolves  $S(t, x)$  on a 4D grid
# * Computes the full numerical Hessian at the grid center
# * Extracts eigenvalues  $\lambda_1(t) \dots \lambda_4(t)$ 
# * Tracks curvature spectrum over RG time
=====

import jax
import jax.numpy as jnp
import numpy as np
from jax import jit

#-----
# Grid setup (4D)
#-----
L = 24          # Reduce if memory tight; increase to 32 if GPU allows
xmax = 2.0
dx = 2*xmax / L

x = jnp.linspace(-xmax, xmax, L)
X, Y, Z, W = jnp.meshgrid(x, x, x, x, indexing="ij")
grid = jnp.stack([X, Y, Z, W], axis=-1)      # shape (L,L,L,L,4)

#-----
# Strongly convex quadratic potential:  $S_0(x) = 0.5 x^T H x$ 
#-----

Hmat = jnp.array([
    [4.0, 0.3, 0.1, 0.0],
    [0.3, 3.0, 0.2, 0.1],
    [0.1, 0.2, 5.0, 0.1],
    [0.0, 0.1, 0.1, 3.5]
])

#  $S_0$  via broadcasting with Einstein summation
S = 0.5 * jnp.einsum('...i,ij,...j->...', grid, Hmat, grid)

#-----
# 4D Laplacian
#-----

```

```

def laplace4(F):
    return (
        -8*F
        + jnp.roll(F, 1, axis=0) + jnp.roll(F, -1, axis=0)
        + jnp.roll(F, 1, axis=1) + jnp.roll(F, -1, axis=1)
        + jnp.roll(F, 1, axis=2) + jnp.roll(F, -1, axis=2)
        + jnp.roll(F, 1, axis=3) + jnp.roll(F, -1, axis=3)
    ) / (dx*dx)

#-----
# Gradient magnitude  $|\nabla S|^2$  in 4D
#-----

def grad2_4(F):
    Fx = (jnp.roll(F, -1, axis=0) - jnp.roll(F, 1, axis=0)) / (2*dx)
    Fy = (jnp.roll(F, -1, axis=1) - jnp.roll(F, 1, axis=1)) / (2*dx)
    Fz = (jnp.roll(F, -1, axis=2) - jnp.roll(F, 1, axis=2)) / (2*dx)
    Fw = (jnp.roll(F, -1, axis=3) - jnp.roll(F, 1, axis=3)) / (2*dx)
    return Fx*Fx + Fy*Fy + Fz*Fz + Fw*Fw

#-----
# HJ time step
#-----

@jit
def hj_step(S, dt):
    return S + dt * (laplace4(S) - grad2_4(S))

#-----
# Full 4x4 Hessian at center via finite differences
#-----

def hessian4_center(S):
    c = L//2

    # Second derivatives along axes
    Sxx = (S[c+1,c,c,c] - 2*S[c,c,c,c] + S[c-1,c,c,c]) / (dx*dx)
    Syy = (S[c,c+1,c,c] - 2*S[c,c,c,c] + S[c,c-1,c,c]) / (dx*dx)
    Szz = (S[c,c,c+1,c] - 2*S[c,c,c,c] + S[c,c,c-1,c]) / (dx*dx)
    Sww = (S[c,c,c,c+1] - 2*S[c,c,c,c] + S[c,c,c,c-1]) / (dx*dx)

    # Mixed derivatives: Sxy, Sxz, Sxw, Syz, Syw, Szw
    Sxy = (S[c+1,c+1,c,c] - S[c+1,c-1,c,c]
           - S[c-1,c+1,c,c] + S[c-1,c-1,c,c]) / (4*dx*dx)

    Sxz = (S[c+1,c,c+1,c] - S[c+1,c,c-1,c]
           - S[c-1,c,c+1,c] + S[c-1,c,c-1,c]) / (4*dx*dx)

    Sxw = (S[c+1,c,c,c+1] - S[c+1,c,c,c-1]
           - S[c-1,c,c,c+1] + S[c-1,c,c,c-1]) / (4*dx*dx)

```

```

Syz = (S[c,c+1,c+1,c] - S[c,c+1,c-1,c]
       - S[c,c-1,c+1,c] + S[c,c-1,c-1,c]) / (4*dx*dx)

Syw = (S[c,c+1,c,c+1] - S[c,c+1,c,c-1]
       - S[c,c-1,c,c+1] + S[c,c-1,c,c-1]) / (4*dx*dx)

Szw = (S[c,c,c+1,c+1] - S[c,c,c+1,c-1]
       - S[c,c,c-1,c+1] + S[c,c,c-1,c-1]) / (4*dx*dx)

Hmat_num = jnp.array([
    [Sxx, Sxy, Sxz, Sxw],
    [Sxy, Syy, Syz, Syw],
    [Sxz, Syz, Szz, Szw],
    [Sxw, Syw, Szw, Sww]
])

evals = np.linalg.eigvalsh(np.array(Hmat_num))
return evals

#-----
# Run semigroup and record eigenvalues
#-----

dt = 0.0005
steps = 300

print("step      λ1      λ2      λ3      λ4   (sorted eigenvalues)")

for t in range(steps):
    S = hj_step(S, dt)

    if t % 30 == 0:
        ev = hessian4_center(S)
        print(f"{t:4d}   {ev[0]:.6f}   {ev[1]:.6f}   {ev[2]:.6f}   {ev[3]:.6f}")

```

step	λ1	λ2	λ3	λ4	(sorted eigenvalues)
0	3.137619	3.801772	4.401150	5.462403	
30	2.866945	3.411298	3.885940	4.689939	
60	2.639317	3.093663	3.478878	4.109222	
90	2.445215	2.830214	3.149128	3.656686	
120	2.277726	2.608154	2.876544	3.294059	
150	2.131735	2.418447	2.647444	2.996953	
180	2.003350	2.254495	2.452184	2.749084	
210	1.889568	2.111381	2.283774	2.539118	
240	1.788015	1.985362	2.137040	2.358979	
270	1.696828	1.873549	2.008032	2.202747	

```
import numpy as np
```

```

# Recorded times
t_vals = np.array([0,30,60,90,120,150,180,210,240,270], dtype=float)

# Recorded eigenvalue curves ( $\lambda_1 < \lambda_2 < \lambda_3 < \lambda_4$ )
lam1 = np.array([3.137619, 2.866945, 2.639317, 2.445215, 2.277726,
                 2.131735, 2.003350, 1.889568, 1.788015, 1.696828])

lam2 = np.array([3.801772, 3.411298, 3.093663, 2.830214, 2.608154,
                 2.418447, 2.254495, 2.111381, 1.985362, 1.873549])

lam3 = np.array([4.401150, 3.885940, 3.478878, 3.149128, 2.876544,
                 2.647444, 2.452184, 2.283774, 2.137040, 2.008032])

lam4 = np.array([5.462403, 4.689939, 4.109222, 3.656686, 3.294059,
                 2.996953, 2.749084, 2.539118, 2.358979, 2.202747])

lam_arrays = [lam1, lam2, lam3, lam4]
alpha_list = []
intercepts = []

print("Eigenvalue   alpha_i           intercept_i (1/ $\lambda_i(0)$ )")
print("-----")

for idx, lam in enumerate(lam_arrays, start=1):

    inv_lam = 1.0 / lam
    A = np.vstack([t_vals, np.ones_like(t_vals)]).T
    alpha_i, intercept_i = np.linalg.lstsq(A, inv_lam, rcond=None)[0]

    alpha_list.append(alpha_i)
    intercepts.append(intercept_i)

    print(f"i={idx}:      {alpha_i:.10f}      {intercept_i:.10f}")

# Optional: compute predicted curves
predicted = [1.0/(intercepts[i] + alpha_list[i]*t_vals) for i in range(4)]

# Print predicted vs true for  $\lambda_1$  (as an example)
print("\nComparison for  $\lambda_1(t)$ :")
for t, true, pred in zip(t_vals, lam1, predicted[0]):
    print(f"t={t:4.0f}   true={true:8.4f}   pred={pred:8.4f}")

```

Eigenvalue	alpha_i	intercept_i (1/ $\lambda_i(0)$)
i=1:	0.0010022789	0.3187423305
i=2:	0.0010026004	0.2630756084
i=3:	0.0010028736	0.2272639797
i=4:	0.0010033081	0.1831387770

Comparison for $\lambda_1(t)$:

t= 0 true= 3.1376 pred= 3.1372

t= 0	true= 3.1570	pred= 3.1570
t= 30	true= 2.8669	pred= 2.8669
t= 60	true= 2.6393	pred= 2.6394
t= 90	true= 2.4452	pred= 2.4453
t= 120	true= 2.2777	pred= 2.2778
t= 150	true= 2.1317	pred= 2.1318
t= 180	true= 2.0034	pred= 2.0034
t= 210	true= 1.8896	pred= 1.8896
t= 240	true= 1.7880	pred= 1.7880
t= 270	true= 1.6968	pred= 1.6968

```

#=====
# GPU-JAX: 4D HJ Flow WITH HAAR-TYPE MASS TERM
#
#  $S_0(x) = 0.5 x^T H x + m^2 * ||x||^2 + \text{beta} * ||x||^4$ 
#
# This models:
#   (1) quadratic Haar curvature ( $m^2 ||A||^2$ )
#   (2) higher-order Haar Jacobian terms ( $\text{beta} ||A||^4$ )
#
# and evolves under viscous Hamilton-Jacobi flow:
#
#  $dS/dt = \Delta S - |\nabla S|^2$ 
#
# Computes full 4x4 Hessian spectrum at center.
#=====

import jax
import jax.numpy as jnp
import numpy as np
from jax import jit

#-----
# 4D grid
#-----
L = 24
xmax = 2.0
dx = 2*xmax / L

x = jnp.linspace(-xmax, xmax, L)
X, Y, Z, W = jnp.meshgrid(x, x, x, x, indexing="ij")
grid = jnp.stack([X, Y, Z, W], axis=-1) # shape (... ,4)

#-----
# Haar-type mass term parameters
#-----
m2 = 1.0 # Haar mass scale
beta = 0.05 # quartic stabilizer

#-----
# Base quadratic anisotropic curvature (as before)

```

```

#-----
Hmat = jnp.array([
    [4.0, 0.3, 0.1, 0.0],
    [0.3, 3.0, 0.2, 0.1],
    [0.1, 0.2, 5.0, 0.1],
    [0.0, 0.1, 0.1, 3.5],
])

#-----
# Initial potential S0(x)
# S0 = 0.5 x^T H x + m^2 ||x||^2 + beta ||x||^4
#-----
r2 = jnp.einsum('...i,...i->...', grid, grid)
quad = 0.5 * jnp.einsum('...i,ij,...j->...', grid, Hmat, grid)
haar_term = m2 * r2
quartic = beta * (r2**2)

S = quad + haar_term + quartic

#-----
# 4D Laplacian & gradient-squared
#-----
def laplace4(F):
    return (
        -8*F
        + jnp.roll(F,1,0)+jnp.roll(F,-1,0)
        + jnp.roll(F,1,1)+jnp.roll(F,-1,1)
        + jnp.roll(F,1,2)+jnp.roll(F,-1,2)
        + jnp.roll(F,1,3)+jnp.roll(F,-1,3)
    ) / (dx*dx)

def grad2_4(F):
    Fx = (jnp.roll(F,-1,0)-jnp.roll(F,1,0))/(2*dx)
    Fy = (jnp.roll(F,-1,1)-jnp.roll(F,1,1))/(2*dx)
    Fz = (jnp.roll(F,-1,2)-jnp.roll(F,1,2))/(2*dx)
    Fw = (jnp.roll(F,-1,3)-jnp.roll(F,1,3))/(2*dx)
    return Fx*Fx + Fy*Fy + Fz*Fz + Fw*Fw

#-----
# HJ time step
#-----
@jit
def hj_step(S, dt):
    return S + dt * (laplace4(S) - grad2_4(S))

#-----
# Full 4x4 Hessian at center
#-----
def hessian4_center(S):
    c = L//2
    Sxx = (S[c+1,c,c,c]-2*S[c,c,c,c]+S[c-1,c,c,c])/(dx*dx)

```

```
Syy = (S[c,c+1,c,c]-2*S[c,c,c,c]+S[c,c-1,c,c])/(dx*dx)
Szz = (S[c,c,c+1,c]-2*S[c,c,c,c]+S[c,c,c-1,c])/(dx*dx)
Sww = (S[c,c,c,c+1]-2*S[c,c,c,c]+S[c,c,c,c-1])/(dx*dx)
```

```
Sxy = (S[c+1,c+1,c,c]-S[c+1,c-1,c,c]
        - S[c-1,c+1,c,c]+S[c-1,c-1,c,c])/(4*dx*dx)
```

```
Sxz = (S[c+1,c,c+1,c]-S[c+1,c,c-1,c]
        - S[c-1,c,c+1,c]+S[c-1,c,c-1,c])/(4*dx*dx)
```

```
Sxw = (S[c+1,c,c,c+1]-S[c+1,c,c,c-1]
        - S[c-1,c,c,c+1]+S[c-1,c,c,c-1])/(4*dx*dx)
```

```
Syz = (S[c,c+1,c+1,c]-S[c,c+1,c-1,c]
        - S[c,c-1,c+1,c]+S[c,c-1,c-1,c])/(4*dx*dx)
```

```
Syw = (S[c,c+1,c,c+1]-S[c,c+1,c,c-1]
        - S[c,c-1,c,c+1]+S[c,c-1,c,c-1])/(4*dx*dx)
```

```
Szw = (S[c,c,c+1,c+1]-S[c,c,c+1,c-1]
        - S[c,c,c-1,c+1]+S[c,c,c-1,c-1])/(4*dx*dx)
```

```
Hnum = jnp.array([
    [Sxx, Sxy, Sxz, Sxw],
    [Sxy, Syy, Syz, Syw],
    [Sxz, Syz, Szz, Szw],
    [Sxw, Syw, Szw, Sww]
])
```

```
return np.linalg.eigvalsh(np.array(Hnum))
```

```
#-----
# Run flow
#-----
dt = 0.0004
steps = 300
```

```
print("step      λ1      λ2      λ3      λ4      (Haar RG)")
for t in range(steps):
    S = hj_step(S, dt)

    if t % 30 == 0:
        ev = hessian4_center(S)
        print(f"{t:4d}    {ev[0]:.6f}    {ev[1]:.6f}    {ev[2]:.6f}    {ev[3]:.6f}")
```

step	λ1	λ2	λ3	λ4	(Haar RG)
0	5.314069	5.979081	6.577921	7.639357	
30	4.730976	5.244640	5.695179	6.466385	
60	4.259698	4.667785	5.018534	5.603515	
90	3.871800	4.203512	4.484058	4.942648	

```

120  3.547456  3.822220  4.051587  4.420572
150  3.272463  3.503689  3.694634  3.997839
180  3.036546  3.233768  3.395149  3.648672
210  2.832010  3.002167  3.140341  3.355417
240  2.653069  2.801350  2.920954  3.105705
270  2.495223  2.625582  2.730120  2.890516

```

```

import numpy as np

# Haar eigenvalues you've recorded:
lam1_H = np.array([5.314069, 4.730976, 4.259698, 3.871800, 3.547456,
                   3.272463, 3.036546, 2.832010, 2.653069, 2.495223])

lam2_H = np.array([5.979081, 5.244640, 4.667785, 4.203512, 3.822220,
                   3.503689, 3.233768, 3.002167, 2.801350, 2.625582])

lam3_H = np.array([6.577921, 5.695179, 5.018534, 4.484058, 4.051587,
                   3.694634, 3.395149, 3.140341, 2.920954, 2.730120])

lam4_H = np.array([7.639357, 6.466385, 5.603515, 4.942648, 4.420572,
                   3.997839, 3.648672, 3.355417, 3.105705, 2.890516])

lam_arrays_H = [lam1_H, lam2_H, lam3_H, lam4_H]

t_vals = np.array([0,30,60,90,120,150,180,210,240,270], float)

print("Eigenvalue    $\alpha_i$ (Haar)       intercept_i")

for i, lam in enumerate(lam_arrays_H, start=1):
    inv_lam = 1.0 / lam
    A = np.vstack([t_vals, np.ones_like(t_vals)]).T
    alpha_i, intercept_i = np.linalg.lstsq(A, inv_lam, rcond=None)[0]
    print(f"i={i}:      {alpha_i:.10f}      {intercept_i:.10f}")

```

```

Eigenvalue    $\alpha_i$ (Haar)       intercept_i
i=1:         0.0007880153      0.1876355891
i=2:         0.0007916689      0.1668507983
i=3:         0.0007939418      0.1517155462
i=4:         0.0007967519      0.1307107120

```

```

#=====
#   GPU-JAX: 4D HJ Flow with Haar Mass + YM-style Quartic Plaquette
#=====

import jax
import jax.numpy as jnp
import numpy as np
from jax import jit

..

```

```

#-----
# GRID (4D)
#-----
L = 24
xmax = 2.0
dx = 2*xmax / L

x = jnp.linspace(-xmax, xmax, L)
X, Y, Z, W = jnp.meshgrid(x, x, x, x, indexing='ij')
grid = jnp.stack([X, Y, Z, W], axis=-1)  # (... ,4)

#-----
# PARAMETERS
#-----
m2      = 1.0      # Haar mass scale
beta    = 0.05     # stabilizer
gamma   = 0.12     # YM-like quartic plaquette coefficient

#-----
# BASE ANISOTROPIC QUADRATIC CURVATURE (as before)
#-----
Hmat = jnp.array([
    [4.0, 0.3, 0.1, 0.0],
    [0.3, 3.0, 0.2, 0.1],
    [0.1, 0.2, 5.0, 0.1],
    [0.0, 0.1, 0.1, 3.5]
])

#-----
# COMPUTE TERMS
#-----
r2 = jnp.einsum('...i,...i->...', grid, grid)
quad = 0.5 * jnp.einsum('...i,ij,...j->...', grid, Hmat, grid)
haar = m2 * r2
quartic = beta * (r2**2)

# YM-like plaquette term:
#  $\sum_{\mu < \nu} x_{\mu}^2 * x_{\nu}^2$ 
x1 = grid[...,0]; x2 = grid[...,1]
x3 = grid[...,2]; x4 = grid[...,3]

YM_plaq = (
    x1*x1 * x2*x2 +
    x1*x1 * x3*x3 +
    x1*x1 * x4*x4 +
    x2*x2 * x3*x3 +
    x2*x2 * x4*x4 +
    x3*x3 * x4*x4
)

S = quad + haar + quartic + gamma * YM_plaq

```

```

#-----
# LAPLACIAN + GRAD2
#-----
def laplace4(F):
    return (
        -8*F +
        jnp.roll(F,1,0)+jnp.roll(F,-1,0) +
        jnp.roll(F,1,1)+jnp.roll(F,-1,1) +
        jnp.roll(F,1,2)+jnp.roll(F,-1,2) +
        jnp.roll(F,1,3)+jnp.roll(F,-1,3)
    ) / (dx*dx)

def grad2_4(F):
    Fx = (jnp.roll(F,-1,0)-jnp.roll(F,1,0))/(2*dx)
    Fy = (jnp.roll(F,-1,1)-jnp.roll(F,1,1))/(2*dx)
    Fz = (jnp.roll(F,-1,2)-jnp.roll(F,1,2))/(2*dx)
    Fw = (jnp.roll(F,-1,3)-jnp.roll(F,1,3))/(2*dx)
    return Fx*Fx + Fy*Fy + Fz*Fz + Fw*Fw

@jit
def hj_step(S, dt):
    return S + dt * (laplace4(S) - grad2_4(S))

#-----
# FULL 4x4 HESSIAN
#-----
def hessian4_center(S):
    c = L//2
    Sxx = (S[c+1,c,c,c]-2*S[c,c,c,c]+S[c-1,c,c,c])/(dx*dx)
    Syy = (S[c,c+1,c,c]-2*S[c,c,c,c]+S[c,c-1,c,c])/(dx*dx)
    Szz = (S[c,c,c+1,c]-2*S[c,c,c,c]+S[c,c,c-1,c])/(dx*dx)
    Sww = (S[c,c,c,c+1]-2*S[c,c,c,c]+S[c,c,c,c-1])/(dx*dx)

    Sxy = (S[c+1,c+1,c,c]-S[c+1,c-1,c,c]
           - S[c-1,c+1,c,c]+S[c-1,c-1,c,c])/(4*dx*dx)
    Sxz = (S[c+1,c,c+1,c]-S[c+1,c,c-1,c]
           - S[c-1,c,c+1,c]+S[c-1,c,c-1,c])/(4*dx*dx)
    Sxw = (S[c+1,c,c,c+1]-S[c+1,c,c,c-1]
           - S[c-1,c,c,c+1]+S[c-1,c,c,c-1])/(4*dx*dx)
    Syz = (S[c,c+1,c+1,c]-S[c,c+1,c-1,c]
           - S[c,c-1,c+1,c]+S[c,c-1,c-1,c])/(4*dx*dx)
    Syw = (S[c,c+1,c,c+1]-S[c,c+1,c,c-1]
           - S[c,c-1,c,c+1]+S[c,c-1,c,c-1])/(4*dx*dx)
    Szw = (S[c,c,c+1,c+1]-S[c,c,c+1,c-1]
           - S[c,c,c-1,c+1]+S[c,c,c-1,c-1])/(4*dx*dx)

    Hnum = jnp.array([
        [Sxx, Sxy, Sxz, Sxw],
        [Sxy, Syy, Syz, Syw],
        [Sxz, Syz, Szz, Szw],
        [Sxw, Syw, Szw, Sww]
    ])

```

```

        [Sxz, Syz, Szz, Szw],
        [Sxw, Syw, Szw, Sww]
    ])

    return np.linalg.eigvalsh(np.array(Hnum))

#-----
# RUN FLOW
#-----
dt = 0.0004
steps = 300

print("step   λ1      λ2      λ3      λ4   (Haar + YM quartic)")
for t in range(steps):
    S = hj_step(S, dt)
    if t % 30 == 0:
        ev = hessian4_center(S)
        print(f"{t:4d}   {ev[0]:.6f}   {ev[1]:.6f}   {ev[2]:.6f}   {ev[3]:.6f}")

```

step	λ1	λ2	λ3	λ4	(Haar + YM quartic)
0	5.317576	5.985277	6.584897	7.648608	
30	4.746849	5.261582	5.712169	6.483884	
60	4.280406	4.688396	5.038575	5.622919	
90	3.893856	4.224702	4.504283	4.961584	
120	3.569202	3.842653	4.070808	4.438169	
150	3.293206	3.522845	3.712478	4.013888	
180	3.055967	3.251467	3.411499	3.663182	
210	2.850043	3.018443	3.155244	3.368526	
240	2.669711	2.816243	2.934525	3.117526	
270	2.510584	2.639218	2.742485	2.901196	

```

import numpy as np

t_vals = np.array([0,30,60,90,120,150,180,210,240,270], float)

lam1_YM = np.array([5.317576, 4.746849, 4.280406, 3.893856, 3.569202,
                    3.293206, 3.055967, 2.850043, 2.669711, 2.510584])

lam2_YM = np.array([5.985277, 5.261582, 4.688396, 4.224702, 3.842653,
                    3.522845, 3.251467, 3.018443, 2.816243, 2.639218])

lam3_YM = np.array([6.584897, 5.712169, 5.038575, 4.504283, 4.070808,
                    3.712478, 3.411499, 3.155244, 2.934525, 2.742485])

lam4_YM = np.array([7.648608, 6.483884, 5.622919, 4.961584, 4.438169,
                    4.013888, 3.663182, 3.368526, 3.117526, 2.901196])

lam_arrays_YM = [lam1_YM, lam2_YM, lam3_YM, lam4_YM]

print("Eigenvalue   αi(YM+Haar)   intercepti")

```

```

for i, lam in enumerate(lam_arrays_YM, start=1):
    inv_lam = 1.0 / lam
    A = np.vstack([t_vals, np.ones_like(t_vals)]).T
    alpha_i, intercept_i = np.linalg.lstsq(A, inv_lam, rcond=None)[0]
    print(f"i={i}:      {alpha_i:.10f}      {intercept_i:.10f}")

```

Eigenvalue	$\alpha_i(\text{YM+Haar})$	intercept_i
i=1:	0.0007799263	0.1871075463
i=2:	0.0007854121	0.1663754420
i=3:	0.0007887369	0.1513088164
i=4:	0.0007928276	0.1303804288

```

#=====
# GPU-JAX: 4D HJ Flow With:
#   * Haar mass term
#   * YM quartic plaquette
#   * SU(2) adjoint-action curvature term
#=====

import jax
import jax.numpy as jnp
import numpy as np
from jax import jit

#-----
# Grid
#-----
L = 24
xmax = 2.0
dx = 2*xmax / L

x = jnp.linspace(-xmax, xmax, L)
X,Y,Z,W = jnp.meshgrid(x,x,x,x, indexing="ij")
grid = jnp.stack([X,Y,Z,W], axis=-1) # (... ,4)

#-----
# Parameters
#-----
m2      = 1.0      # Haar mass
beta    = 0.05     # stabilizer
gamma   = 0.12     # YM quartic plaquette
lambda_adj = 0.5   # SU(2) adjoint curvature coefficient

#-----
# Base quadratic
#-----
Hmat = jnp.array([
    [4.0, 0.3, 0.1, 0.0],
    [0.3, 3.0, 0.2, 0.1],

```

```

        [0.1, 0.2, 5.0, 0.1],
        [0.0, 0.1, 0.1, 3.5]
    ])

r2 = jnp.einsum('...i,...i->...', grid, grid)
quad = 0.5 * jnp.einsum('...i,ij,...j->...', grid, Hmat, grid)

# Haar mass term
haar = m2 * r2

# quartic stabilizer
quartic = beta * (r2**2)

# YM-like quartic plaquette
x1,x2,x3,x4 = grid[...,0],grid[...,1],grid[...,2],grid[...,3]
YM_plaq = (x1*x1*x2*x2 + x1*x1*x3*x3 + x1*x1*x4*x4 +
           x2*x2*x3*x3 + x2*x2*x4*x4 + x3*x3*x4*x4)

# SU(2) adjoint curvature: 2*(A1^2 + A2^2 + A3^2)
adj_term = lambda_adj * (2*(x1*x1 + x2*x2 + x3*x3))

# Full initial potential
S = quad + haar + quartic + gamma*YM_plaq + adj_term

#-----
# Laplacian and grad2
#-----
def laplace4(F):
    return (
        -8*F +
        jnp.roll(F,1,0)+jnp.roll(F,-1,0) +
        jnp.roll(F,1,1)+jnp.roll(F,-1,1) +
        jnp.roll(F,1,2)+jnp.roll(F,-1,2) +
        jnp.roll(F,1,3)+jnp.roll(F,-1,3)
    ) / (dx*dx)

def grad2_4(F):
    Fx = (jnp.roll(F,-1,0)-jnp.roll(F,1,0))/(2*dx)
    Fy = (jnp.roll(F,-1,1)-jnp.roll(F,1,1))/(2*dx)
    Fz = (jnp.roll(F,-1,2)-jnp.roll(F,1,2))/(2*dx)
    Fw = (jnp.roll(F,-1,3)-jnp.roll(F,1,3))/(2*dx)
    return Fx*Fx + Fy*Fy + Fz*Fz + Fw*Fw

@jit
def hj_step(S, dt):
    return S + dt*(laplace4(S) - grad2_4(S))

#-----
# 4x4 Hessian extraction
#-----
def hessian4_center(S):

```

```

def hessian4_center(S):
    c = L//2

    Sxx = (S[c+1,c,c,c]-2*S[c,c,c,c]+S[c-1,c,c,c])/(dx*dx)
    Syy = (S[c,c+1,c,c]-2*S[c,c,c,c]+S[c,c-1,c,c])/(dx*dx)
    Szz = (S[c,c,c+1,c]-2*S[c,c,c,c]+S[c,c,c-1,c])/(dx*dx)
    Sww = (S[c,c,c,c+1]-2*S[c,c,c,c]+S[c,c,c,c-1])/(dx*dx)

    Sxy = (S[c+1,c+1,c,c]-S[c+1,c-1,c,c]
           - S[c-1,c+1,c,c]+S[c-1,c-1,c,c])/(4*dx*dx)
    Sxz = (S[c+1,c,c+1,c]-S[c+1,c,c-1,c]
           - S[c-1,c,c+1,c]+S[c-1,c,c-1,c])/(4*dx*dx)
    Sxw = (S[c+1,c,c,c+1]-S[c+1,c,c,c-1]
           - S[c-1,c,c,c+1]+S[c-1,c,c,c-1])/(4*dx*dx)
    Syz = (S[c,c+1,c+1,c]-S[c,c+1,c-1,c]
           - S[c,c-1,c+1,c]+S[c,c-1,c-1,c])/(4*dx*dx)
    Syw = (S[c,c+1,c,c+1]-S[c,c+1,c,c-1]
           - S[c,c-1,c,c+1]+S[c,c-1,c,c-1])/(4*dx*dx)
    Szw = (S[c,c,c+1,c+1]-S[c,c,c+1,c-1]
           - S[c,c,c-1,c+1]+S[c,c,c-1,c-1])/(4*dx*dx)

    Hnum = jnp.array([
        [Sxx,Sxy,Sxz,Sxw],
        [Sxy,Syy,Syz,Syw],
        [Sxz,Syz,Szz,Szw],
        [Sxw,Syw,Szw,Sww]
    ])
    return np.linalg.eigvalsh(np.array(Hnum))

#-----
# RUN
#-----
dt = 0.0004
steps = 300

print("step   λ1    λ2    λ3    λ4    (Haar + YM + SU(2) adjoint)")
for t in range(steps):
    S = hj_step(S, dt)
    if t % 30 == 0:
        ev = hessian4_center(S)
        print(f"{t:4d}    {ev[0]:.6f}  {ev[1]:.6f}  {ev[2]:.6f}  {ev[3]:.6f}")

```

step	λ1	λ2	λ3	λ4	(Haar + YM + SU(2) adjoint)
0	5.969544	7.494316	8.735322	9.790115	
30	5.248125	6.374480	7.238564	7.939785	
60	4.676118	5.540572	6.175710	6.674832	
90	4.213378	4.897046	5.383110	5.756155	
120	3.832201	4.386069	4.769803	5.059031	
150	3.513237	3.970883	4.281417	4.512133	
180	3.242647	3.627025	3.883405	4.071715	
210	3.010325	3.337685	3.552908	3.709481	

```
240 2.808814 3.090909 3.274122 3.406350
270 2.632397 2.877986 3.035824 3.148988
```

```
import numpy as np

t_vals = np.array([0,30,60,90,120,150,180,210,240,270], float)

lam1_SU2 = np.array([5.969544, 5.248125, 4.676118, 4.213378, 3.832201,
                    3.513237, 3.242647, 3.010325, 2.808814, 2.632397])

lam2_SU2 = np.array([7.494316, 6.374480, 5.540572, 4.897046, 4.386069,
                    3.970883, 3.627025, 3.337685, 3.090909, 2.877986])

lam3_SU2 = np.array([8.735322, 6.175710, 6.175710, 5.383110, 4.769803,
                    4.281417, 3.883405, 3.552908, 3.274122, 3.035824])

lam4_SU2 = np.array([9.790115, 7.939785, 6.674832, 5.756155, 5.059031,
                    4.512133, 4.071715, 3.709481, 3.406350, 3.148988])

# Pack them
lam_arrays_SU2 = [lam1_SU2, lam2_SU2, lam3_SU2, lam4_SU2]

print("Eigenvalue    $\alpha_i$ (SU2+Haar+YM)   intercept_i")
print("-----")

alpha_SU2 = []
intercept_SU2 = []

for i, lam in enumerate(lam_arrays_SU2, start=1):
    inv_lam = 1.0 / lam
    A = np.vstack([t_vals, np.ones_like(t_vals)]).T
    alpha_i, intercept_i = np.linalg.lstsq(A, inv_lam, rcond=None)[0]
    alpha_SU2.append(alpha_i)
    intercept_SU2.append(intercept_i)
    print(f"i={i}:      {alpha_i:.10f}      {intercept_i:.10f}")
```

```
Eigenvalue    $\alpha_i$ (SU2+Haar+YM)   intercept_i
-----
i=1:      0.0007875071      0.1668187001
i=2:      0.0007932955      0.1330191860
i=3:      0.0007627501      0.1211391406
i=4:      0.0007980805      0.1019825721
```

```
#=====
# 4D Geometric RG Flow (Viscous Hamilton-Jacobi) with:
#
#   ✓ Strongly convex quadratic base (anisotropic)
#   ✓ Haar mass term
#   ✓ YM-style quartic plaquette term
```

```

#  ✓ SU(2) adjoint curvature
#  ✓ SU(3) adjoint curvature (Casimir = 3)
#
#  PDE:   dS/dt = ΔS - |∇S|^2
#
#  Output: full 4x4 Hessian spectrum over RG time
#=====

import jax
import jax.numpy as jnp
import numpy as np
from jax import jit

#-----
# 4D grid
#-----
L = 24                # reduce if memory is low
xmax = 2.0
dx = 2*xmax / L

x = jnp.linspace(-xmax, xmax, L)
X, Y, Z, W = jnp.meshgrid(x, x, x, x, indexing="ij")
grid = jnp.stack([X, Y, Z, W], axis=-1) # (... ,4)

#-----
# Parameter set
#-----
m2          = 1.0      # Haar mass term
beta        = 0.05     # stabilizer for quartic ||x||^4
gamma_YM    = 0.12     # YM-like plaquette coupling
lambda_SU2  = 0.5      # SU(2) adjoint curvature coefficient
lambda_SU3  = 0.4      # SU(3) adjoint curvature coefficient (Casimir)
C2_SU3      = 3.0      # SU(3) adjoint Casimir eigenvalue

#-----
# Base anisotropic quadratic curvature
#-----
Hmat = jnp.array([
    [4.0, 0.3, 0.1, 0.0],
    [0.3, 3.0, 0.2, 0.1],
    [0.1, 0.2, 5.0, 0.1],
    [0.0, 0.1, 0.1, 3.5]
])

#-----
# Radial and quadratic terms
#-----
r2 = jnp.einsum('...i,...i->...', grid, grid)
quad = 0.5 * jnp.einsum('...i,ij,...j->...', grid, Hmat, grid)

haar = m2 * r2

```

```

quad = m2 * r2
quartic = beta * (r2**2)

#-----
# YM-style quartic plaquette (commutator surrogate)
#-----
x1, x2, x3, x4 = grid[...,0], grid[...,1], grid[...,2], grid[...,3]

YM_plaq = (
    x1*x1*x2*x2 +
    x1*x1*x3*x3 +
    x1*x1*x4*x4 +
    x2*x2*x3*x3 +
    x2*x2*x4*x4 +
    x3*x3*x4*x4
)

#-----
# SU(2) adjoint curvature:  $2\|A\|^2$  (A only first 3 coords)
#-----
adj_SU2 = lambda_SU2 * (2*(x1*x1 + x2*x2 + x3*x3))

#-----
# SU(3) adjoint curvature:  $C2 = 3 \rightarrow 3\|A\|^2$  (use full 4-vector slice)
#-----
adj_SU3 = lambda_SU3 * (C2_SU3 * r2)

#-----
# Total initial potential
#-----
S = quad + haar + quartic + gamma_YM*YM_plaq + adj_SU2 + adj_SU3

#-----
# 4D Laplacian
#-----
def laplace4(F):
    return (
        -8*F +
        jnp.roll(F,1,0) + jnp.roll(F,-1,0) +
        jnp.roll(F,1,1) + jnp.roll(F,-1,1) +
        jnp.roll(F,1,2) + jnp.roll(F,-1,2) +
        jnp.roll(F,1,3) + jnp.roll(F,-1,3)
    ) / (dx*dx)

#-----
#  $|\nabla S|^2$  in 4D
#-----
def grad2_4(F):
    Fx = (jnp.roll(F,-1,0) - jnp.roll(F,1,0)) / (2*dx)
    Fy = (jnp.roll(F,-1,1) - jnp.roll(F,1,1)) / (2*dx)
    Fz = (jnp.roll(F,-1,2) - jnp.roll(F,1,2)) / (2*dx)

```

```

    Fw = (jnp.roll(F,-1,3) - jnp.roll(F,1,3)) / (2*dx)
    return Fx*Fx + Fy*Fy + Fz*Fz + Fw*Fw

#-----
# HJ step
#-----
@jit
def hj_step(S, dt):
    return S + dt*(laplace4(S) - grad2_4(S))

#-----
# 4x4 Hessian at center
#-----
def hessian4_center(S):
    c = L//2

    Sxx = (S[c+1,c,c,c] - 2*S[c,c,c,c] + S[c-1,c,c,c])/(dx*dx)
    Syy = (S[c,c+1,c,c] - 2*S[c,c,c,c] + S[c,c-1,c,c])/(dx*dx)
    Szz = (S[c,c,c+1,c] - 2*S[c,c,c,c] + S[c,c,c-1,c])/(dx*dx)
    Sww = (S[c,c,c,c+1] - 2*S[c,c,c,c] + S[c,c,c,c-1])/(dx*dx)

    Sxy = (S[c+1,c+1,c,c] - S[c+1,c-1,c,c] -
           S[c-1,c+1,c,c] + S[c-1,c-1,c,c])/(4*dx*dx)

    Sxz = (S[c+1,c,c+1,c] - S[c+1,c,c-1,c] -
           S[c-1,c,c+1,c] + S[c-1,c,c-1,c])/(4*dx*dx)

    Sxw = (S[c+1,c,c,c+1] - S[c+1,c,c,c-1] -
           S[c-1,c,c,c+1] + S[c-1,c,c,c-1])/(4*dx*dx)

    Syz = (S[c,c+1,c+1,c] - S[c,c+1,c-1,c] -
           S[c,c-1,c+1,c] + S[c,c-1,c-1,c])/(4*dx*dx)

    Syw = (S[c,c+1,c,c+1] - S[c,c+1,c,c-1] -
           S[c,c-1,c,c+1] + S[c,c-1,c,c-1])/(4*dx*dx)

    Szw = (S[c,c,c+1,c+1] - S[c,c,c+1,c-1] -
           S[c,c,c-1,c+1] + S[c,c,c-1,c-1])/(4*dx*dx)

    Hnum = jnp.array([
        [Sxx, Sxy, Sxz, Sxw],
        [Sxy, Syy, Syz, Syw],
        [Sxz, Syz, Szz, Szw],
        [Sxw, Syw, Szw, Sww]
    ])
    return np.linalg.eigvalsh(np.array(Hnum))

#-----
# Time evolution
#-----
dt = 0.0001

```

```

dt = 0.0001
steps = 300

print("step   λ1    λ2    λ3    λ4    (Haar + YM + SU(2) + SU(3))")
for t in range(steps):
    S = hj_step(S, dt)
    if t % 30 == 0:
        ev = hessian4_center(S)
        print(f"{t:4d}    {ev[0]:.6f}    {ev[1]:.6f}    {ev[2]:.6f}    {ev[3]:.6f}")

```

step	λ1	λ2	λ3	λ4	(Haar + YM + SU(2) + SU(3))
0	8.552138	10.070476	11.306206	12.356505	
30	7.111890	8.121508	8.897969	9.529510	
60	6.082116	6.800884	7.333221	7.754219	
90	5.310836	5.848202	6.235627	6.536125	
120	4.712193	5.128965	5.423448	5.648622	
150	4.234345	4.566936	4.798306	4.973281	
180	3.844222	4.115753	4.302308	4.442171	
210	3.519747	3.745613	3.899191	4.013550	
240	3.245703	3.436505	3.565157	3.660408	
270	3.011178	3.174494	3.283816	3.364371	

```

import numpy as np

t_vals = np.array([0,30,60,90,120,150,180,210,240,270], float)

lam1_SU3 = np.array([8.552138, 7.111890, 6.082116, 5.310836, 4.712193,
                     4.234345, 3.844222, 3.519747, 3.245703, 3.011178])

lam2_SU3 = np.array([10.070476, 8.121508, 6.800884, 5.848202, 5.128965,
                     4.566936, 4.115753, 3.745613, 3.436505, 3.174494])

lam3_SU3 = np.array([11.306206, 8.897969, 7.333221, 6.235627, 5.423448,
                     4.798306, 4.302308, 3.899191, 3.565157, 3.283816])

lam4_SU3 = np.array([12.356505, 9.529510, 7.754219, 6.536125, 5.648622,
                     4.973281, 4.442171, 4.013550, 3.660408, 3.364371])

lam_arrays_SU3 = [lam1_SU3, lam2_SU3, lam3_SU3, lam4_SU3]

print("Eigenvalue   αi(SU3)           intercepti")
print("-----")

alpha_SU3 = []
intercept_SU3 = []

for i, lam in enumerate(lam_arrays_SU3, start=1):
    inv = 1.0 / lam
    A = np.vstack([t_vals, np.ones_like(t_vals)]).T
    alpha_i, b_i = np.linalg.lstsq(A, inv, rcond=None)[0]
    alpha_SU3.append(alpha_i)

```

```
alpha_SU3.append(alpha_i,
intercept_SU3.append(b_i)
print(f"i={i}:      {alpha_i:.10f}      {b_i:.10f}")
```

Eigenvalue	$\alpha_i(\text{SU3})$	intercept_i
i=1:	0.0007973450	0.1166651489
i=2:	0.0007992039	0.0991425524
i=3:	0.0008004338	0.0883685728
i=4:	0.0008011904	0.0809022973

```
#=====
# GPU-JAX: SU(3) COMMUTATOR CURVATURE TERM FOR 4D HJ RG FLOW
#
# Adds:
#      S_SU3_comm(A) = kappa * sum_{a,b,c} f_{abc}^2 A_b^2 A_c^2
#
# where A = (x1,x2,x3,x4,0,0,0,0) ∈ su(3)_adj
#
# Plug-and-play with your existing S = ... definition.
#=====

import jax
import jax.numpy as jnp
import numpy as np
import math

#-----
# 1. Build full SU(3) f_{abc} structure constant tensor (8x8x8)
#-----
def build_su3_f_tensor():
    f = np.zeros((8,8,8), dtype=float)

    entries = [
        (1,2,3, 1.0),
        (1,4,7, 0.5),
        (1,5,6, 0.5),
        (2,4,6, 0.5),
        (2,5,7,-0.5),
        (3,4,5, 0.5),
        (3,6,7, 0.5),
        (4,5,8, math.sqrt(3)/2),
        (6,7,8, math.sqrt(3)/2),
    ]

    # Fill antisymmetric f_{abc}
    for (a,b,c,val) in entries:
        a-=1; b-=1; c-=1 # 0-index
        f[a,b,c] = val
        f[b,c,a] = val
```

```

        f[c,a,b] = val
        f[a,c,b] = -val
        f[c,b,a] = -val
        f[b,a,c] = -val

    return jnp.array(f)

f_su3 = build_su3_f_tensor()

#-----
# 2. Construct SU(3)-commutator curvature term for 4D grid
#-----

def su3_commutator_curvature(grid, kappa):
    """
    grid[...,4] is 4D coordinate mapped to A1..A4,
    remaining adjoint components set to 0.
    A_b = A[b] in su(3) adjoint; b=0..7
    """

    # A has shape (... ,8)
    A = jnp.concatenate(
        [grid, jnp.zeros(grid.shape[:-1] + (4,))],
        axis=-1
    ) # shape (... ,8)

    # A_b^2 and A_c^2
    A2 = A * A

    # Compute: S = kappa * sum_{a,b,c} f_{abc}^2 * A_b^2 * A_c^2
    # We don't need a-loop explicitly; it's absorbed as a sum of f^2 over a.
    f2 = f_su3 * f_su3 # square each entry

    # contraction over b,c but sum over a included:
    # S(x) = kappa * sum_{b,c} ( sum_a f[a,b,c]^2 ) * A_b^2 * A_c^2
    M = jnp.sum(f2, axis=0) # shape (8,8)

    return kappa * jnp.einsum('...b,...c,bc->...', A2, A2, M)

```

```
S = quad + haar + quartic + gamma_YM*YM_plaq + adj_SU2 + adj_SU3_mass + su3_cc
```

Next steps:

[Explain error](#)